PGINN step by step: simulate, train and test the networks for one regime

Peter Beerli

October 23, 2026

Abstract

The script `../code/reproduce.sh` rebuilds every figure and table of the paper, but it calls more than 50 Python scripts, and the core of the method is hard to see among them. This tutorial follows the core path alone, for one regime: 10 loci per data set and n=10 samples. It simulates genealogies under the sweep model, turns them into network inputs, trains one network with the MSE loss and one with the PGINN loss, and tests both on new simulated data. The commands are collected in `run_tutorial.sh`; the whole run takes about 70 s on a laptop with 16 cores. The script uses the paper's priors, sizes and random seeds, and its data and networks are identical, bit for bit, to the paper's files for this regime (Section 8).

Contents

1 Overview **1**

2 Step 1: one genealogy by hand **2**

3 Step 2: training, validation and test data **4**

4 Step 3: from genealogies to network inputs **4**

5 Step 4: training **5**

6 Step 5: test of recovery **6**

7 Step 6: estimates for new data **7**

8 Check against the paper's files **8**

9 Further: the rest of the code **8**

1 Overview

Run all steps, or one step at a time, from any directory:

```
cd tutorial                  # this folder; the scripts are in ../code
./run_tutorial.sh            # all steps, about 70 s
./run_tutorial.sh simulate   # one step; later steps need the earlier ones
./run_tutorial.sh check      # compare with the paper's data and networks
```

Step	Command	Script(s)	Output in `work/`
1	`genealogy`	`sweep.py`, `simtree.py`	`tp.txt`, `one.json`, `genealogy.pdf`
2	`simulate`	`simulate_sweep.py` → `simtree.py`	`train-sweep/` and three more folders
3	`features`	`prepare.py`	`*-features.npy`, `*-params.npy`, `*-ages.npy`
4	`train`	`train_pginn.py`	`model-{mse,pginn}.pt`, `scaler-*.npz`
5	`test`	`compare.py`	`recovery.pdf`, `recovery.csv`
6	`score`	`train_pginn.predict()`	printed summary
–	`check`	–	comparison with `../code/data`, `../code/model`

Table 1: The steps of `run_tutorial.sh`. All scripts are in `../code/`; paths in this document are relative to the folder `tutorial/`.

The script needs the Python packages of `../requirements.txt` (`numpy`, `torch`, `matplotlib`); `msprime` and MIGRATE are not needed. Everything is written to `work/` (about 70 MB).

The model in brief. A single population has the mutation-scaled size $\theta{=}c\mu$ and the population-scaled selection coefficient $s{=}c\,s_g$. A completed hard sweep shrinks the ancestral class of the sample toward the past. Toward the past, the frequency of the selected allele falls as

$$x(t;s) = \frac{x_0}{x_0 + (1-x_0)\,e^{st}}, \qquad x_0 = 0.999, \tag{1}$$

down to the floor $x_{\min}{=}10^{-4}$. Genealogies measure time in expected mutations per site, $u{=}\theta t$. On this scale the size of the selected class and the coalescent rate for k lineages are

$$\theta_{\mathrm{e}}(u) = \theta\, x(u/\theta; s), \qquad \lambda_k(u) = \frac{k(k-1)}{\theta_{\mathrm{e}}(u)}. \tag{2}$$

The network estimates (θ, s) from the genealogies of a data set.

2 Step 1: one genealogy by hand

The simulator `simtree.py` draws coalescent genealogies with a piecewise-constant θ. A sweep enters only through a *timepoints* file: each line holds an age u and the value of θ_{e} from that age on. `sweep.py` writes this file. It places 40 breakpoints evenly in $\log x$ between $0.999\,x_0$ and $x_{\min}$ and converts each to its age by inverting Eq. 1:

`../code/sweep.py`, `make_timepoints_file` (core lines)

```
x_points = np.geomspace(x0 * 0.999, x_min, n_segments)
t_points = t_of_x(x_points, rate_per_age(s, theta), x0)
with open(path, "w") as f:
    for t, x in zip(t_points, x_points):
        f.write(f"{t:.10g} {theta * x:.10g}\n")
```

`rate_per_age(s, theta)` is s/θ, so the trajectory is evaluated directly on the age scale u. Step 1 of the script writes the schedule for $\theta{=}0.01$, $s{=}100$ and simulates one genealogy with it and one neutral genealogy:

```
python -c "import sweep; sweep.make_timepoints_file('tp.txt', 0.01, 100.0)"
python simtree.py -t 0.01 -i 10 -a 1.0 -sel 100 -tp tp.txt -l 1 --json -f one.json --
    seed 1
python simtree.py -t 0.01 -i 10 -a 1.0 -l 1 --json -f one-neutral.json --seed 1
```

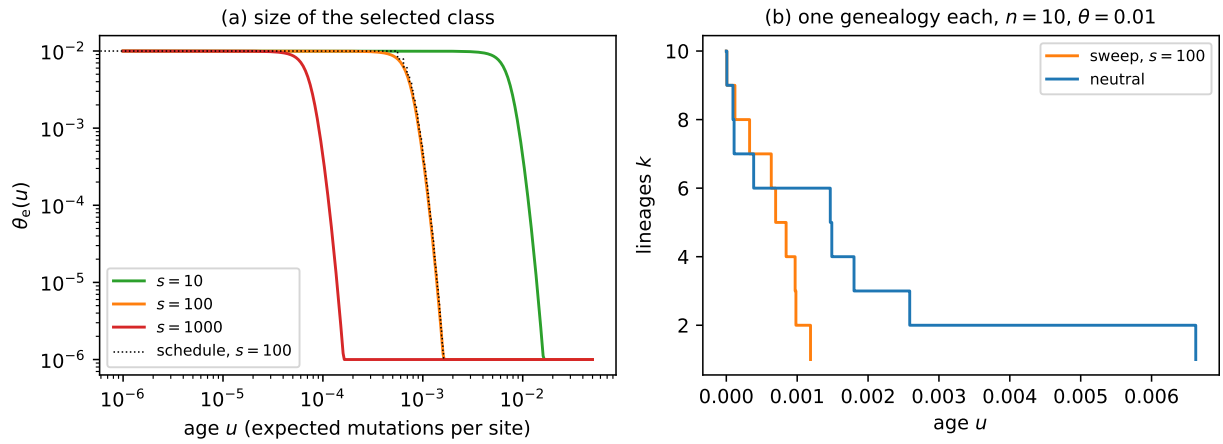


Figure 1: (a) $\theta_e(u)$ at θ=0.01 for three values of s (Eq. 2), and the 40-step schedule that `simtree.py` receives for s=100 (dotted). (b) Number of lineages against age for the two genealogies of step 1 (same random seed). Made by `genealogy_fig.py`.

The options are: `-t` θ, `-i` sample size, `-a 1.0` the Kingman coalescent (the simulator also has a fractional coalescent, α<1; the default `"1.0,1.0"` would set up two populations), `-l` number of loci, `-json` the output format used below, and `-seed` the random seed. The value of `-sel` is only recorded in the output; the sweep itself acts through `-tp`. The first lines of `tp.txt` are

```
6.936475559e-05 0.00998001
0.0005593159182 0.007881142072
0.0006407142897 0.006223681175
```

Figure 1a shows the schedule against the continuous $\theta_e(u)$; Figure 1b shows the two genealogies. The sweep genealogy has its root at u=0.0012, where θ_e has collapsed; the neutral genealogy has its root at u=0.0066.

The JSON output. `one.json` is a list with one entry per locus. Each entry has the fields

Field	Length (n=10)	Content
`params`	–	$[[\theta], [\alpha], [s]]$
`tree`	–	the genealogy in Newick format; tips `0_0` … `0_9` (population_tip)
`ages`	9	ages of the $n-1$ internal nodes, oldest first
`age`	45	age of the most recent common ancestor (MRCA) of each pair of tips
`inter`	45	populations of the two tips of each pair (always `[0,0]` here)
`M`	55	per pair: number of edges from the root to the MRCA; per tip: 1
`m`	55	per pair: length of the path from the root to the MRCA; per tip: its branch length

The vectors `M` and `m` are the Kendall–Colijn tree vectors, computed in `../code/Mm.py`. Note that the JSON names are swapped relative to `Mm.py`, where `m` is the edge count and `M` the path length; the JSON field `m` holds the lengths. Pairs are ordered by tip name, tips are listed after the pairs.

3 Step 2: training, validation and test data

`simulate_sweep.py` draws (θ, s) from the prior and calls `simtree.py` once per draw, in parallel on all cores. For $s>0$ it writes a timepoints file as in step 1. Each draw has its own random seed, the base seed plus the draw number, so a data set is reproducible exactly.

```
PRIOR=(--theta-min 0.001 --theta-max 0.05 --theta-prior loguniform --s-prior loguniform
     --s-min 1)
python simulate_sweep.py -f train-sweep     -n 1000 -l 10 --nind 10 --smax 1000 --seed
    100000 $PRIOR --s-zero-frac 0.25
python simulate_sweep.py -f test-sweep      -n 100  -l 10 --nind 10 --smax 1000 --seed
    110000 $PRIOR --s-zero-frac 0.25
python simulate_sweep.py -f predict-sweep   -n 400  -l 10 --nind 10 --smax 1000 --seed
    120000 $PRIOR
python simulate_sweep.py -f predict-neutral -n 400  -l 10 --nind 10 --smax 0    --seed
    130000 $PRIOR
```

The four sets are

Folder	Draws	Prior on s	Use
`train-sweep`	1000	0 with prob. 0.25, else log-uniform on $[1, 1000]$	training
`test-sweep`	100	as training	validation (choice of the epoch)
`predict-sweep`	400	log-uniform on $[1, 1000]$	test of recovery
`predict-neutral`	400	s=0	test under neutrality

In all four sets θ is log-uniform on $[0.001, 0.05]$. The file names follow `../code/reproduce.sh`, where the validation set is called `test-sweep`. Each draw is one data set of 10 independent loci (`-l 10`), stored as one JSON file, for example `train-sweep/test-000001-theta-0.002846-s-3.7509.json`. Each folder also has a `manifest.csv` with the parameters and seed of every draw. In the training set 262 of the 1000 draws have s=0.

4 Step 3: from genealogies to network inputs

`prepare.py` reads every JSON file of a folder and writes three arrays, one row per draw (the counts are for n=10):

```
python prepare.py --train train-sweep --interest 0,2 --prefix train-sweep
#  train-sweep-features.npy (1000, 225)   network input
#  train-sweep-params.npy   (1000, 2)     targets: [log theta, s]
#  train-sweep-ages.npy     (1000, 9)     mean internal-node ages, for the PGINN loss
```

`-interest 0,2` selects entries 0 and 2 of the flattened `params`, that is θ and s. The program prints "Saved 3 averaged samples"; the 3 is the number of arrays, not the number of draws.

The feature vector. Within each genealogy, the pair entries and the tip entries are sorted by age, so that the vectors of different genealogies can be averaged (tip labels have no meaning across independent loci). The feature vector is then the mean over the 10 loci of four blocks. With P=$n(n-1)/2 + n$=55 entries per pair-and-tip block:

Columns	Block	Content	Scale in the network
0–54	0	path lengths, root to MRCA (JSON `m`)	log
55–109	1	edge counts, root to MRCA (JSON `M`)	linear
110–164	2	pair MRCA ages, then the tip branch lengths	log
165–224	3	internal-node ages, youngest first, padded to $\max(20, n)$ rows $\times$ 3	log (age column)

The rows of block 3 are $(\text{age}, \text{pop}_1, \text{pop}_2)$; for one population the last two columns are zero, and rows beyond the $n-1$ internal nodes are zero padding. In the paper's main networks the age-valued entries are replaced by $\log(x + 10^{-5})$. This transformation is the function `featlog` in `../code/feature_ages.py`. `train_pginn.py` applies it when called with `-input-transform featlog` and stores the choice in the scaler file, so the raw `-features.npy` serve as input throughout. (`reproduce.sh` also runs `feature_ages.py`, which writes `-featlog.npy` and other input variants to disk for the comparisons in the supplement; the tutorial does not need them.)

Targets and ages. `-params.npy` holds $[\log\theta, s]$ with s on its natural scale; `train_pginn.py` turns the second column into $\log(1+s)$. `-ages.npy` holds, for every draw, the internal-node ages of each locus sorted from the oldest to the youngest, averaged over the loci. The PGINN loss evaluates the coalescent rates at these ages.

5 Step 4: training

```
b=(--x-train train-sweep-features.npy --y-train train-sweep-params.npy --ages-train
    train-sweep-ages.npy
   --x-val test-sweep-features.npy --y-val test-sweep-params.npy --ages-val test-sweep-
    ages.npy
   --input-transform featlog --seed 67)
python train_pginn.py $b --pginnratio 0   --out-model model-mse.pt   --out-scaler
    scaler-mse.npz
python train_pginn.py $b --pginnratio 0.1 --out-model model-pginn.pt --out-scaler
    scaler-pginn.npz
```

Network. A multilayer perceptron with two hidden layers of 256 units, each linear layer followed by ReLU and batch normalization, and a linear output layer with two values (`MLPRegressor`). Inputs and targets are standardized with the means and standard deviations of the training set. Training uses AdamW (learning rate 10^{-3}, weight decay 10^{-4}), batches of 64 and 300 epochs.

Loss. The network predicts $\hat{\phi}=(\log\hat{\theta}, \log(1+\hat{s}))$. The training loss mixes the mean squared error with the PGINN penalty, with weight w (`-pginnratio`):

$$\mathcal{L} = (1-w)\,\text{MSE} + w\,\mathcal{L}_{\text{local}}, \qquad \mathcal{L}_{\text{local}} = \frac{1}{K}\sum_{k=2}^{n}\left[\log\frac{\hat{\lambda}_k(u_{(k)})}{\lambda_k(u_{(k)})} + \frac{\lambda_k(u_{(k)})}{\hat{\lambda}_k(u_{(k)})} - 1\right], \tag{3}$$

with $K=n-1$ and $u_{(k)}$ the age at which k lineages merge into $k-1$. $w=0$ is the MSE network, and $w=0.1$ the PGINN network of the paper. The penalty is the KL divergence between two exponential distributions with the rates under the predicted and the true parameters, each evaluated at the observed event age. In the code:

../code/train_pginn.py, coalescent_kl_selection (core lines)

```
K = ages.shape[1]
k = torch.arange(2, K + 2, device=ages.device, dtype=ages.dtype)  # [2,3,...,K+1]
k_km1 = k * (k - 1)  # (K,)

x_hat = x_trajectory_torch(ages, (s_hat / theta_hat).unsqueeze(1))     # (B,K)
x_true = x_trajectory_torch(ages, (s_true / theta_true).unsqueeze(1))  # (B,K)

theta_eff_hat = theta_hat.unsqueeze(1) * x_hat + eps
theta_eff_true = theta_true.unsqueeze(1) * x_true + eps

lam_hat = (k_km1.unsqueeze(0) / theta_eff_hat).clamp(max=MAX_LAMBDA)
lam = (k_km1.unsqueeze(0) / theta_eff_true).clamp(max=MAX_LAMBDA)

kl_per_interval = torch.log(lam_hat / lam + eps) + lam / lam_hat - 1.0
return kl_per_interval.mean(dim=1)
```

Before these lines the function undoes the standardization of the predicted and the true targets and converts them to $\hat{\theta}, \hat{s}$ and θ, s. Column 0 of `ages` is the root, so it belongs to k=2. The mixing of the two terms is in `train_epoch`:

../code/train_pginn.py, train_epoch

```
pred = model(xb)
mse = mse_criterion(pred, yb) if s_weight == 1.0 else weighted_mse(pred, yb, s_weight)
kl_loss = term(pred, yb, y_mean, y_std, ages).mean()
loss = (1.0 - pginnratio) * mse + pginnratio * kl_loss
```

The option `-loss` selects the other penalties of the paper (`exact`, `fisher`, `process`, `process-fwd`); they need other age files (Section 9).

Choice of the epoch and the output. After every epoch the loss on the validation set is computed; the network of the epoch with the lowest validation loss is kept. The last lines of the two training logs (`work/train-*.log`) are

```
==> work/train-mse.log <==
Best validation loss: 0.06665127277374268 at epoch 5
==> work/train-pginn.log <==
Best validation loss: 0.07293154954910279 at epoch 300
```

For this training run the MSE network is the one of epoch 5: its validation loss fluctuates strongly in the first epochs (0.075, 0.081, 0.067, 0.097, 0.108 in epochs 3–7) and does not fall below the value of epoch 5 later; at epoch 300 it is 0.080. Each run writes two files: the network weights (`.pt`) and a scaler file (`.npz`) with the standardization constants, the $\log(1+s)$ link and the input transformation. Both files are needed for prediction. The weights are reproducible bit for bit only with the same random seed and the same number of CPU threads (`-threads`, default 2).

6 Step 5: test of recovery

```
python compare.py --x predict-sweep-features.npy --y predict-sweep-params.npy \
  --model-mse model-mse.pt --scaler-mse scaler-mse.npz \
```

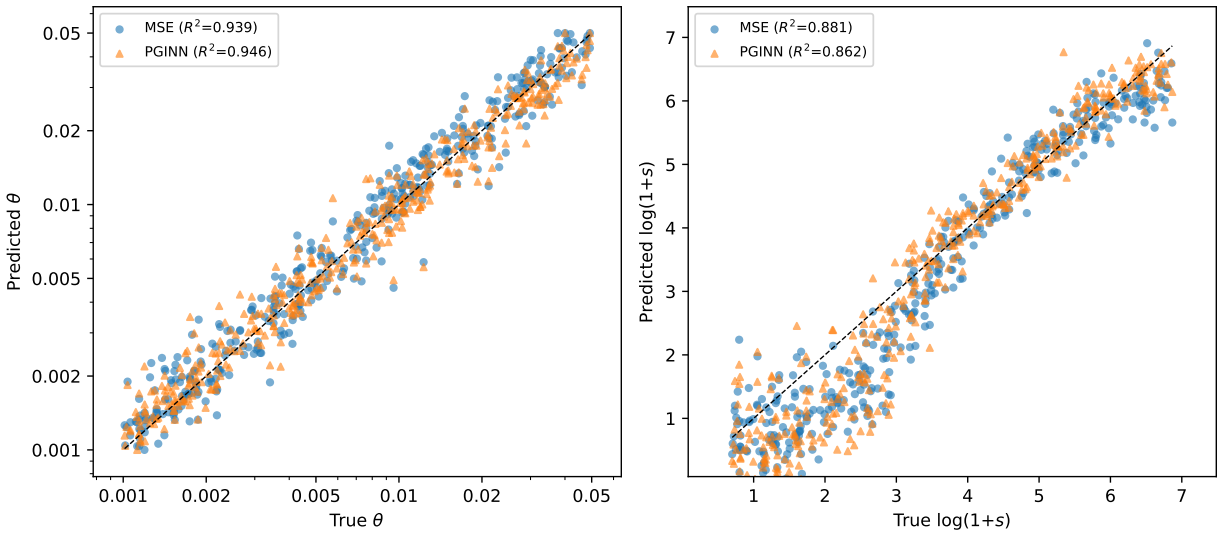


Figure 2: Estimated against true θ and $\log(1+s)$ for the 400 test data sets, MSE and PGINN network (`compare.py`).

```
  --model-pginn model-pginn.pt --scaler-pginn scaler-pginn.npz \
  --out-fig recovery.pdf --out-table recovery.csv
```

`compare.py` estimates (θ, s) for the 400 test data sets with both networks and reports bias, root mean squared error and R^2 (Table 2, Figure 2). s is evaluated on the $\log(1+s)$ scale, the scale of its prior and of the network's target. `reproduce.sh` writes the same table to

Parameter	Loss	Bias	RMSE	R^2
θ	MSE	0.00041	0.0030	0.939
	PGINN	−0.00046	0.0028	0.946
$\log(1+s)$	MSE	−0.338	0.622	0.881
	PGINN	−0.290	0.671	0.862

Table 2: Recovery on the 400 test data sets (`work/recovery.csv`, rounded). The file also has the rows for s on its natural scale.

`../figures/recovery_10l_featlog.csv`. It is one training run on one data set. The comparison of the losses in `nested_eval.py` uses 10 independent data sets with three training runs each.

7 Step 6: estimates for new data

New data enter the networks as feature vectors with the layout of step 3. The function `predict` in `train_pginn.py` applies the scaler, runs the network and undoes the links:

`../code/train_pginn.py`, `predict` (core lines)

```
scaler = np.load(scaler_path, allow_pickle=True)
if "input_transform" in scaler and str(scaler["input_transform"]) == "featlog":
    from feature_ages import featlog
    X = featlog(X)
```

```
x_std = (X - scaler["x_mean"]) / scaler["x_std"]
model = MLPRegressor(X.shape[1], scaler["y_mean"].shape[0])
model.load_state_dict(torch.load(model_path, map_location="cpu"))
model.eval()
with torch.no_grad():
    z = model(torch.from_numpy(x_std).float()).numpy() * scaler["y_std"] + scaler["
    y_mean"]
theta = np.clip(np.exp(z[:, 0]) + MIN_THETA, *THETA_SUPPORT)
s = np.clip(np.expm1(z[:, 1]), *S_SUPPORT)
```

Estimates are bounded to the support of the training prior, $\theta \in [0.001, 0.05]$ and $s \in [0, 1000]$. Step 6 scores the 400 neutral test data sets:

```
mse    median s_hat   1.31   s_hat < 5: 0.96   corr(log theta_hat, log theta) 0.992
pginn  median s_hat   1.06   s_hat < 5: 0.95   corr(log theta_hat, log theta) 0.993
```

Under neutrality both networks give small $\hat{s}$ for most data sets. The paper uses $\hat{s}$ as the detection statistic, with a threshold from the neutral sets of another data set (`nested_eval.py`). For real sequence data the genealogies are not known; the paper scores the posterior genealogies of MIGRATE (`real_adh_migrate_scan.py`); that script writes each tree in the JSON format of step 1 (with `Mm.py`) and builds the feature vectors with the code of `prepare.py`.

8 Check against the paper's files

`./run_tutorial.sh check` compares the arrays of step 3 and the networks of step 4 with the paper's files:

```
work/{set}-{features,params,ages}.npy  vs  ../code/data/{set}-10l-{features,params,ages
    }.npy
work/model-mse.pt                      vs  ../code/model/model-mse-10l-featlog.pt
work/model-pginn.pt                    vs  ../code/model/model-pginn-10l-0.1-featlog.pt
```

The networks are those of training run 67, created by the `simulate` and `revision-train` stages of `reproduce.sh` (or fetched from Zenodo with `fetch_data.sh`). On 2026-10-23 all twelve arrays and both networks were identical. If they differ on another machine, the most likely reasons are a different number of CPU threads or a different version of `torch` or `numpy`.

9 Further: the rest of the code

Other regimes. `reproduce.sh` runs steps 2–4 for six regimes, with the same commands and other values of `-l`, `-nind` and the seeds:

Regime	Loci	n	Base seed
`10l`	10	10	100000
`1l`	1	10	200000
`10l-n20`	10	20	300000
`1l-n20`	1	20	400000
`1l-n176`	1	176	600000
`1l-n100`	1	100	800000

The validation, selected test and neutral test sets use the base seed plus 10000, 20000 and 30000. The paper trains 10 runs per regime and loss (seeds 67–76). The regime `1l-n176` is used for the *Adh* data and `1l-n100` for the simulated chromosomes (stage `chrom-simulate`).

Other penalties. The penalties of the paper's penalty family differ in the option `-loss` and in the file passed as `-ages-*`:

Penalty (paper)	`-loss`	`-ages-train`, `-ages-val`
local rate divergence	`local` (default)	`-ages.npy`
exact log-likelihood ratio	`exact`	`-locusages.npy` (`per_locus_ages.py`)
process divergences	`process`, `process-fwd`	`-locusages.npy`
Fisher penalty	`fisher`	`-fisher.npy` (`fisher_info.py`)
weighted MSE	`-pginnratio 0 -s-weight 5`	`-ages.npy` (not used)

Map of the scripts. The other scripts of `../code/` fall into these groups; the file `../reproducibility.txt` lists which script writes which output file.

Group	Scripts
Core (this tutorial)	`sweep.py`, `simtree.py` (with `tree.py`, `Mm.py`), `simulate_sweep.py`, `prepare.py`, `feature_ages.py`, `train_pginn.py`, `compare.py`
Inputs for other penalties	`per_locus_ages.py`, `fisher_info.py`
Choice of the weights	`lambda_sweep_val.py`, `variant_sweep_val.py`, `w1_diagnostic.py`, `w1_check.py`
Comparison of the losses	`nested_eval.py` (main table), `variance_mechanism.py`, `combined_detector.py`, `eval_n176.py`, `penalty_share.py`, `loss_timing.py`, `asymmetry_fig.py`
Baselines	`mle_baseline.py`, `mle_check.py`, `ages_regression.py`, `star_ratio_appendix.py`
Checks of the model and the penalties	`calibrate_sweep.py`, `discretization_check.py`, `fisher_check.py`, `process_kl_check.py`, `neutral_limit.py`
`msprime` chromosomes	`msprime_scan*.py`, `msprime_metrics.py`, `robustness_demography.py`
Chromosomes through MIGRATE	`chrom_pipeline.sh`, `chrom_sweep_sim.py`, `chrom_*.py`, `average_runs.py`
Adh data	`four_gamete_test.py`, `make_migrate_windows.py`, `extract_dgrp_region.py`, `real_adh_migrate_scan.py`, `adh_*.py`, `real_data_summary_fig.py`, `mcdonald_kreitman_test.py`
Other	`detect_selection.py`, `paired_test.py`, `../talk/growth_scan_talk.py` (growth scan)